



SBOM Document Quality Guide

Compliance Management Guide for the Supply Chain

An official guide published by the OpenChain Project (www.openchainproject.org)

Version: 2026.04.03

Table of Contents:

- [0. Preface](#)
- [1. Scope and SBOM Document Quality](#)
- [2. Terms and Definitions](#)
- [3. Guidelines to Enhance SBOM Document Quality](#)
 - [3.1. Data Format](#)
 - [3.1.1 Verification and reference material](#)
 - [3.1.2 Rationale](#)
 - [3.2 Elements to be included](#)
 - [3.2.1 Verification and reference material](#)
 - [3.2.2 Rationale](#)
 - [3.3 File Format](#)
 - [3.3.1 Verification and reference material](#)
 - [3.3.2 Rationale](#)
 - [3.4 Timing of SBOM Document delivery](#)
 - [3.4.1 Verification and reference material](#)
 - [3.4.2 Rationale](#)
 - [3.5 SBOM Document Scope](#)
 - [3.5.1 Verification and reference material](#)
 - [3.5.2 Rationale](#)
 - [3.6 SBOM Document Verification](#)
 - [3.6.1 Verification and reference material](#)
 - [3.6.2 Rationale](#)
 - [3.7 SBOM Document Confidentiality](#)
 - [3.7.1 Verification and reference material](#)
 - [3.7.2 Rationale](#)
- [4. Conformant Notice](#)
- [5. Common SBOM Document Quality Issues and Improvement Measures](#)
 - [5.1 Ensuring Accurate and Consistent “Value” Information](#)
 - [5.1.1 Issue Overview](#)
 - [5.1.2 Detailed Description](#)
 - [5.1.3 Improvement Measures](#)
 - [5.1.4 Risks and Considerations](#)
 - [5.2 Standardization and Normalization of Component Granularity](#)
 - [5.2.1 Issue Overview](#)
 - [5.2.2 Detailed Description](#)
 - [5.2.3 Improvement Measures](#)
 - [5.2.4 Risks and Considerations](#)
 - [5.3 Complementing Source Code Information and Enhancing Transparency](#)
 - [5.3.1 Issue Overview](#)
 - [5.3.2 Detailed Description](#)
 - [5.3.3 Improvement Measures](#)
 - [5.3.4 Risks and Considerations](#)

<u>5.4 Component Identification Issues Affecting Vulnerability Handling and License Compliance</u>	
<u>5.4.1 Issue Overview</u>	
<u>5.4.2 Detailed Description</u>	
<u>5.4.3 Improvement Measures</u>	
<u>5.4.4 Risks and Considerations</u>	
<u>5.5 Establishing a Tamper Detection and Change Management System</u>	
<u>5.5.1 Issue Overview</u>	
<u>5.5.2 Detailed Description</u>	
<u>5.5.3 Improvement Measures</u>	
<u>5.5.4 Risks and Considerations</u>	
<u>5.6 Clarifying the Scope of Descriptions and Defining Accountability</u>	
<u>5.6.1 Issue Overview</u>	
<u>5.6.2 Detailed Description</u>	
<u>5.6.3 Improvement Measures</u>	
<u>5.6.4 Risks and Considerations</u>	
<u>5.7 Unified Expression of Component Relationships</u>	
<u>5.7.1 Issue Overview</u>	
<u>5.7.2 Detailed Description</u>	
<u>5.7.3 Improvement Measures</u>	
<u>5.7.4 Risks and Considerations</u>	
<u>5.8 Interoperability and Flexibility Among Tools</u>	
<u>5.8.1 Issue Overview</u>	
<u>5.8.2 Detailed Description</u>	
<u>5.8.3 Improvement Measures</u>	
<u>5.8.4 Risks and Considerations</u>	
<u>6. Compliance Elements for SPDX and CycloneDX in Relation to Legislation and Guidelines</u>	
<u>6.1 References</u>	
<u>6.2 Cross-Regulation Comparison Table</u>	
<u>6.3 NTIA - The Minimum Elements For a Software Bill of Materials (SBOM)</u>	
<u>6.3.1 Required Elements</u>	
<u>6.4 CISA - 2025 Minimum Elements for a Software Bill of Materials (SBOM)</u>	
<u>6.4.1 Required Elements</u>	
<u>6.4.2 Table of Minimum Elements Data Fields</u>	
<u>6.5 BSI TR-03183 Cyber Resilience Requirements for Manufacturers and Products Part 2: Software Bill of Materials (SBOM) Version 2.1.0</u>	
<u>6.5.1 Required Elements</u>	
<u>6.6 OpenChain Telco SBOM Guide Version 1.1</u>	
<u>6.6.1 Required Elements</u>	
<u>6.6.1.1 Document Creation Information</u>	
<u>6.6.1.2 Package Information</u>	
<u>Appendix.1 Acknowledgments</u>	
<u>Appendix.2 Version History</u>	

0. Preface

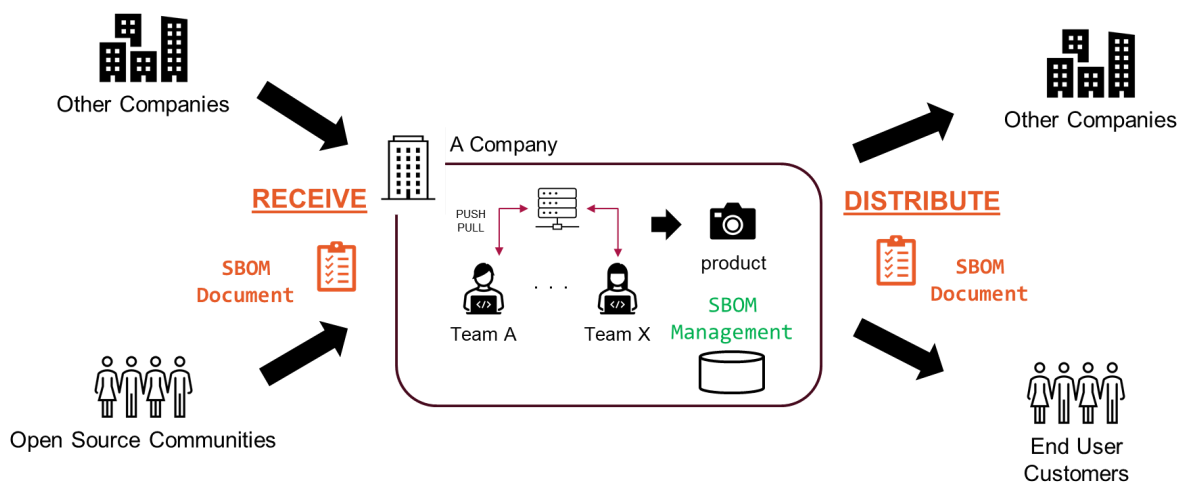
The "OpenChain SBOM Document Quality Guide" is a format-independent framework focused on the quality of the information contained within the document, such as its accuracy and integrity. It defines the essential quality requirements for achieving robust security assurance and license compliance, providing actionable steps to ensure the reliability of the content.

Key considerations and differences when adapting the Telco SBOM Guide to develop this guide:

- **Compatibility:** This guide is designed for broad compatibility beyond the "OpenChain Telco SBOM Guide". By conforming to this guide, an SBOM document not only meets the requirements of the "OpenChain Telco SBOM Guide" but also aligns with various other industry guidelines and regulatory standards.
- **Applicability:** This guide serves as a foundational quality standard applicable across all industries. Its language and requirements have been carefully refined to ensure universal relevance, making it a basic framework for any sector implementing SBOM Document.
- **Format Independence:** This guide is written to be independent of any specific SBOM Data format.
- **Quality Definition:** A new chapter discusses what constitutes a high-quality SBOM Document, explains its importance, and describes how such documents can be effectively utilized.
- **Best practices:** Guidance addressing various challenges in creating and managing SBOM Documents have been incorporated.
- **Practical Examples:** As part of these best practices, practical SBOM Document samples are provided in JSON format along with their corresponding schema.

1. Scope and SBOM Document Quality

While the term "**SBOM**" generally refers to the information that constitutes a software's composition, this guide specifically focuses on the quality of the "**SBOM Document**". In this guide, "**SBOM Document**" is a structured artifact – typically formatted in JSON and based on specifications such as SPDX or CycloneDX – that is exchanged between software distributors and recipients.



This guide, "OpenChain SBOM Document Quality Guide", establishes a clear framework for document quality – centered on security assurance and license compliance – and providing actionable requirements to achieve it. Specifically, documents are evaluated based on following two essential aspects:

- Adequacy of Security Assurance
Assesses whether sufficient baseline information is provided to support an investigation that validates the software's security posture, even if, at the time of delivery, the document does not comprehensively cover all risks, vulnerabilities, or mitigation strategies.
- Effectiveness of License Compliance
Assesses whether the necessary licensing details and usage terms for each software component are properly captured to ensure compliance with relevant laws and regulations.

By adhering to this guide, stakeholders can ensure that the SBOM Documents exchanged within the software supply chain consistently meet high-quality standards.

2. Terms and Definitions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [\[RFC2119\]](#) [\[RFC8174\]](#) when, and only when, they appear in all capitals, as shown here.

Terms	Definitions
Data Format	Data Format means the data format of the information in the SBOM. Possible Data Formats include SPDX, Cyclone DX, SWID, or other proprietary formats.
Entity	Entity shall mean the legal entity (for profit, non profit, or natural person) that distributes software to third parties (e.g., other organizations or individuals). Entity does not include other group companies, or companies under common control of the Entity.
SBOM	A Software Bill of Materials (SBOM) is a formal record containing the details and supply chain relationships of various components used in building software.
SBOM Type	An SBOM can be of one of the following types: Design, Source, Build, Analyzed, Deployed, Runtime. The definition of these types can be found in the CISA document .
SPDX	SPDX (System Package Data Exchange) is the ISO standard (ISO/IEC 5962:2021) for exchanging SBOM for a given software package, including associated license and copyright information. The standard was created by the Linux Foundation's SPDX project .
CycloneDX	CycloneDX is the ECMA standard (ECMA-424) for a full-stack Bill of Materials (BOM) standard that provides advanced supply chain capabilities for cyber risk reduction. The standard was created by the OWASP Foundation, which is a nonprofit foundation for improving software security.
OpenChain Specification ISO/IEC 5230:2020	ISO/IEC 5230:2020 is an international standard that specifies the key requirements of a quality open source license compliance program in order to provide a benchmark that builds trust between organizations exchanging software solutions that

	incorporate open source software. The OpenChain standard is produced by the OpenChain project of the Linux Foundation.
OpenChain Specification ISO/IEC 18974:2023	ISO/IEC 18974:2023 is an international standard from the OpenChain Project that provides requirements for open source software security assurance. It aims to improve software supply chain confidence by managing publicly known security vulnerabilities. Organizations can demonstrate compliance through self-certification or audits.
Transitive dependencies	Transitive dependencies are all components that are necessary for the software to run. They include any dependency of the package that is not a direct dependency.
Package URL(PURL)	Package URL (PURL) is a de facto standard to uniquely identify software packages.
SBOM Document	A Software Bill of Materials (SBOM) document is the output of SBOM information in formats like JSON or YAML for the purpose of accurate information transfer between organizations.
File Format	File Format means the format of SBOM Document. Possible File Formats include JSON, YAML, Excel Sheet etc.
Software Package	A software package is a distributable unit that can consist of a single software component, such as code or a library, or a bundle of related components, including configuration files. It may also include information about dependencies and versioning, making installation, updates, and integration with other systems more efficient. This packaging approach helps streamline software development and maintenance processes.

3. Guidelines to Enhance SBOM Document Quality

Chapter 3 defines the criteria by which the OpenChain SBOM Document Quality Guide recognizes an SBOM Document as high-quality.

3.1. Data Format

An SBOM Document SHALL be provided in a machine-readable format that enables automated processing by organizations, customers, and regulatory or supervisory authorities.

The use of internationally recognized formats such as SPDX® or CycloneDX is therefore considered mandatory for such purposes.

3.1.1 Verification and reference material

SPDX®: <https://spdx.dev/use/specifications/> , <https://tools.spdx.org/app/validate/>

CycloneDX: <https://cyclonedx.org/specification/overview/> ,
<https://github.com/CycloneDX/sbom-utility>

3.1.2 Rationale

To ensure simplified handling and streamlining of tooling and competences in the software supply chain, both for suppliers and consumers of software, SBOM Documents shall employ a format generally recognized as the standard specification for SBOMs as standardized. By harmonizing the use of this standard SBOM Data Format in an organization's external interfaces, the reduction in complexity leads to fewer errors and, as a result, improved the document quality.

3.2 Elements to be included

An SBOM Document SHALL include all elements defined in this chapter, which the OpenChain SBOM Document Quality Guide considers indicative of high quality.

SBOM Document Information

- Data Format Version
Include the SBOM Document format and version to ensure tool interoperability.
- License for the SBOM Document
- Unique ID for the SBOM Document
Include a global unique identifier in the SBOM Document to represent the exchanged SBOM Document. The specification for this unique value varies depending on the Data Format;
For example, CycloneDX has 'serialNumber', while in SPDX version 3 the 'SPDXID' for the SpdxDocument must be global unique and SPDX version 2.X uses a

combination of a globally unique 'DocumentNamespace' and 'SPDXID' to ensure uniqueness.

- Creation information for the SBOM Document

It is essential to include the details of who created the SBOM Document and when, as this information is necessary for contacting the responsible entities in case of issues.

- Author of SBOM Document

If permitted by the respective Data Format specifications, the creator's information should ideally contain a valid email address or a URL for a contact form to ensure they are reachable.

- Creation Date and Time of SBOM Document

- Generation Context

In an SBOM Document, element values specifying the timing and method of its creation SHOULD be included, as these details are crucial for vulnerability management. Therefore, including the generation context such as [SBOM Types defined by CISA](#) in the document can enhance its overall quality.

However, since a standardized element for generation context may not exist, harmonizing their usage across the supply chain is essential.

Software Package information

- Information that can uniquely identify a software package

For accurate license compliance and vulnerability assessment, it is essential to correctly identify each software package. The document shall include the following elements to ensure proper identification:

- Package Name

- Package Version

- Details regarding the package identifiers, such as the [SoftWare Hash Identifiers \(SWHID\)](#), the [Package-URL \(PURL\)](#), [CPE name](#), [gitoid](#) or the URL of the package distribution site, indicating where the corresponding software package can be obtained.

It is also important for both the supplier and the recipient to agree on which fields are most effective for identifying software packages within the SBOM document.

- Package Supplier information

This element should contain contact details for the supplier of the package. It is desirable to include a valid email address or a URL to a contact form, ensuring that inquiries about the package can be addressed promptly.

- Package Hash value

Each Software Package listed in the SBOM Document should include at least one corresponding hash value from the distributed component. This is necessary to verify that the particular component is indeed present in the distribution. The hash algorithm must be chosen from those specified in the respective Data Format and used to calculate the hash of the corresponding source code or binary component.

- Proprietary Software indicator

For proprietary software, elements like Package Name and Package Version may be custom-defined. However, the SBOM Document should clearly indicate the software is proprietary - for example, using package comments or a purl - and should include Package Supplier information to facilitate inquiries.

License information

- License declared in the Software Package
The SBOM Document shall include the license information declared by the distributed package. The declared license SHALL utilize standardized identifiers, for example, [the SPDX License Expressions](#), to ensure clarity and consistency. For proprietary licenses, which are often custom-defined, please provide the contact information for inquiries along with the previously mentioned Proprietary Software indicator.
- License concluded by the Package Distributor
The SBOM Document should include the license determined by the package distributor. For example, when a package is dual-licensed, the relationship with other packages may dictate which license applies. In addition, considering both the declared license and the overall software structure, the distributor must conclude the license for distribution. It is recommended that the distributor include such element in the SBOM Document. If the appropriate license cannot be concluded - and if allowed by the applicable specifications - the element may either be omitted or marked as a 'known unknown', using a placeholder such as 'NOASSERTION' in SPDX document.

Relationship information

- As specified by NTIA SBOM Minimum Elements, the document must specify the primary component of the top level Software Package and containment relationships, indicating that one component contains another.
For more practical details, please refer to Section [“5.6 Clarifying the Scope of Descriptions and Defining Accountability”](#).

Mandatory information required by Specifications and Guidelines

When the SBOM document is used for legal or regulatory compliance purposes, the following requirements MUST be strictly observed.

- Other mandatory Elements as specified in each Data Format
Include all Elements that are mandatory for each Data Format. If the appropriate value for any element is unknown, fill it in with a placeholder such as 'NOASSERTION' in SPDX document, indicating a 'known unknown'. Since each Entity typically processes the SBOM Document using different tools, including these required Elements enhances interoperability and ultimately improves the quality of SBOM Documents across the entire supply chain.
Refer to [“6. Compliance Elements for SPDX and CycloneDX in Relation to Legislation and Guidelines”](#) for more details.
- Other mandatory Elements as specified by the applicable specifications and guidelines that each Entity conforms
Include all Elements that are mandatory for each specification and guideline to which each Entity conforms. If the appropriate value for any element is unknown, fill it in with a placeholder such as 'NOASSERTION' in SPDX document, indicating a 'known unknown'.
When a unified specification across the entire supply chain is not mandated, it is important to clearly indicate which specification the SBOM Document conforms to.

This improves the document's quality by explicitly defining the compliance requirements.

3.2.1 Verification and reference material

The Minimum Elements For a Software Bill of Materials(SBOM) :

<https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>

2025 Minimum Elements for a Software Bill of Materials (SBOM):

<https://www.cisa.gov/resources-tools/resources/2025-minimum-elements-software-bill-materials-sbom>

SPDX®: <https://spdx.dev/use/specifications/>

CycloneDX: <https://cyclonedx.org/specification/overview/>

SWHID: <https://www.swhid.org/>

PURL: <https://github.com/package-url/purl-spec>

3.2.2 Rationale

Clearly defining across the entire supply chain which elements, such as SBOM Document Creation information and Package information details, must be included and distributed helps prevent gaps in the distributed data. Furthermore, clarifying the expected values for each element reduces ambiguous language and inter-tool variations.

3.3 File Format

An SBOM document SHALL be in a machine-readable format, such as those supported by SCA tools. In addition, to facilitate manual review (e.g., for license verification), the document SHOULD be provided in, or be easily convertible to, a human-readable format.

3.3.1 Verification and reference material

XML 1.0: <https://www.w3.org/TR/xml/>

JSON-LD 1.1: <https://www.w3.org/TR/json-ld11/>

ECMA-404: The JSON data interchange syntax:

<https://ecma-international.org/publications-and-standards/standards/ecma-404/>

YAML™: <https://yaml.org/spec/>

For other formats, please refer to the respective specifications.

3.3.2 Rationale

As noted in NTIA SBOM Minimum Elements, when SBOM Documents are exchanged across the supply chain, they must be provided in a machine-readable standard format to ensure interoperability among the various tools used by different entities. At the same time, since SBOM Documents may also be reviewed manually (for example, checking license

information), it is equally important that the document is available in, or can be easily converted to, a human-readable format.

3.4 Timing of SBOM Document delivery

The SBOM Document SHALL be available no later than at the time of the delivery of the software (in either binary or source form).

3.4.1 Verification and reference material

The Minimum Elements For a Software Bill of Materials(SBOM), “Distribution and Delivery”:
<https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>

3.4.2 Rationale

An SBOM Document SHOULD be available no later than at the delivery of the software to ensure that the receiving entity can ingest the software and its SBOM.

3.5 SBOM Document Scope

The SBOM SHALL contain all open source software that is delivered with the product. The SBOM SHOULD contain all commercial components.

If some components are either not included or ambiguous, they SHALL be reported as “Known Unknowns”. When reporting “Known Unknowns”, the SBOM Document SHOULD distinguish between two categories:

1. Genuinely Unknown: Information that is not available at the time of SBOM creation (e.g., a transitive dependency whose details cannot be determined).
2. Intentionally Withheld: Information that exists but is deliberately omitted (e.g., proprietary component details subject to confidentiality agreements).

Clearly labeling the reason for omission helps downstream recipients assess risk accurately and take appropriate action.

Refer to the “[5.6 Clarifying the Scope of Descriptions and Defining Accountability](#)” for more details.

3.5.1 Verification and reference material

The Minimum Elements For a Software Bill of Materials(SBOM), “Known Unknowns”:
<https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>

3.5.2 Rationale

It might not be possible, advisable or feasible to have the commercial component information in the SBOM Document. However, the SBOM Document should convey as complete information as possible.

3.6 SBOM Document Verification

A digital signature of the SBOM Document SHOULD be provided in order to guarantee its integrity.

3.6.1 Verification and reference material

Sigstore: <https://www.sigstore.dev/>

3.6.2 Rationale

While the need to attach a digital signature to the SBOM Document for ensuring its integrity is currently under discussion, few practical implementations exist. Consequently, this topic will be revisited in the future, taking into account the associated costs and practical considerations.

3.7 SBOM Document Confidentiality

Contents of the SBOM Document may be subject to confidentiality agreements. Appropriate access controls SHOULD be implemented as necessary. However, confidentiality agreements and access controls SHALL NOT prevent a recipient from redistributing the SBOM Document along with the applicable software components. Use of the Traffic Light Protocol (TLP) as implemented by CycloneDX 1.7 is a sensible approach for managing this need.

3.7.1 Verification and reference material

The Minimum Elements For a Software Bill of Materials(SBOM), "Access Control":
<https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>

3.7.2 Rationale

Information related to proprietary software may be subject to confidentiality agreements and thus cannot always be public. In such cases, it is acceptable either to restrict access solely to authorized entities within the supply chain or to substitute the information with "Known Unknowns."

However, when these components are distributed through complex supply chains, confidentiality agreements and access controls SHALL NOT restrict SBOM Document redistribution, thereby preventing the loss of critical information during the distribution.

4. Conformant Notice

You MAY use the following statement to indicate that the software components you distribute are accompanied by SBOM Document conforming to the OpenChain SBOM Document Quality Guide:

“This software is provided with SBOM Document defined as high quality by the OpenChain SBOM Quality Guide, which is available at <https://github.com/OpenChain-Project/SBOM-wg/XXX>”

5. Common SBOM Document Quality Issues and Improvement Measures

This section highlights the challenges often encountered when generating and managing SBOM Document, and introduces best practices for addressing them. These measures are designed to enhance the accuracy, consistency, and transparency of SBOM Document, as well as to improve the overall processes involved in handling them.

5.1 Ensuring Accurate and Consistent “Value” Information

5.1.1 Issue Overview

Challenges exist in the inconsistent representation of information such as package names, versions, and supplier names across different companies and tools. Without unified standards, automatic analysis of SBOM Document or vulnerability matching becomes challenging, leading to inaccuracies.

5.1.2 Detailed Description

In many SBOM guides and standards, the element ‘keys’ defining what should be included are explicitly specified along with the corresponding ‘value’ ranges and representation methods. However, since a precise format for these values is often not defined, issues can arise during practical implementation.

For example, the two SBOMs below use different package names - one as 'hello' and the other as 'hello 0.0.1'. This discrepancy is due to differences in the output formats of the tools used.

- [example7-bin.spdx.json](#)
"name": "hello",
- [hello-dist.spdx.json](#)
"name": "hello 0.0.1",

Although verifying that these represent the same package is possible – for example, by comparing their PURLs – the fact that different tools may output different values for the same element name frequently leads to confusion and poses challenges to the smooth operation of SBOM management.

5.1.3 Improvement Measures

- The SBOM Document shall include the items documented in "[3.2 SBOM Elements](#) - Package information, Information that can uniquely identify a software package", ensuring their consistent use throughout the entire supply chain.
- Additionally, since the identifiers used to uniquely identify a package may differ depending on the entity distributing the SBOM Document, it is necessary to also provide information on which combination of elements and corresponding values should be used to accurately identify the software package.

5.1.4 Risks and Considerations

- If the documentation rules are inadequately defined or overly complex, there is a risk of misclassification due to incomplete handling of exceptional cases or unique notations.
- If the rules deviate from actual operational practices, there is a risk of misclassification resulting from either insufficient or excessively stringent checks.
- It should be noted that complete automation of tools and processes is challenging; therefore, final checks and exception handling will require manual review.

5.2 Standardization and Normalization of Component Granularity

5.2.1 Issue Overview

In SBOM terminology, 'granularity' refers to the level of detail at which software components are described — primarily either at the file level or the package level. When SBOM Documents exchanged across the supply chain use different levels of granularity, it becomes difficult to describe and interpret dependency relationships consistently.

This is especially true in complex supply chains where the entity that ultimately creates the SBOM cannot affect the entire supply chain.

5.2.2 Detailed Description

Consider a simplified three-party scenario: **Vendor1** (a component supplier), **Maker1** (a product manufacturer), and **User1** (an end user). If **Vendor1** supplies a file-level SBOM but **Maker1** requires a package-level SBOM, **Maker1** must perform the conversion. The reverse situation creates the same burden. Such conversion requires a full review of the SBOM, increasing workload for the receiving entity.

The assumptions are as follows:

- **Vendor1** provides 'application A' binary along with SBOM Document to **Maker1**
- **Maker1** supplies 'product X' (which includes application A and other OSS) along with SBOM Document to **User1**
- 'application A' depends on the OSS used in product X.

Under these conditions, the following table summarizes the possible combinations of component granularity in the SBOM Document.

Table 5.2.2

		SBOM Document for 'product X' to User1	SBOM Document for 'product X' to User1
	component granularity	file-level	package-level
SBOM Document for 'application A' from Vendor1	file-level	All dependency shall be recorded and provided at the file-level.	Maker1 needs to convert file-level information to package-level for application A.
SBOM Document for 'application A' from Vendor1	package-level	Maker1 needs to decompose package-level information and convert them to file-level.	All dependency shall be recorded and provided at the package-level.

Differences in component granularity within SBOM Document require a full review to determine its granularity level, leading to an increased workload for the integrating entity.

5.2.3 Improvement Measures

- When creating an SBOM, explicitly state within the document whether it is at the file level or the package level. Where the requesting party is able to specify the required granularity level, this should be agreed upon in advance.
 - If the SBOM requestor can specify the granularity, it should. However, in the case of a complex supply chain, there may be situations where the specification from the requester cannot affect the entire supply chain.
- Define explicit granularity elements in future versions of the Data format specifications to clarify whether the information is at the file or package level.

5.2.4 Risks and Considerations

- There is a potential for issues when linking with vulnerability information. For example, if component granularity is mixed, it may not be possible to automatically identify the corresponding vulnerability information, leading to concerns over increased manual effort and time.

5.3 Complementing Source Code Information and Enhancing Transparency

5.3.1 Issue Overview

Software is often distributed in binary form (compiled executables). If the corresponding source code is not recorded in the SBOM Document, it becomes difficult to:

- Verify that the appropriate license terms have been correctly applied
- Assess whether known vulnerabilities affect the exact source code used, or whether they have already been remediated

Including source code information in the SBOM Document significantly improves transparency and traceability throughout the software supply chain.

5.3.2 Detailed Description

Adhering to "[3.2 SBOM Elements](#) – Package information, Information that can uniquely identify a software package" can sometimes allow you to locate source code in repositories like GitHub. However, when creating binary components, patches are often applied for customization or to fix vulnerabilities and bugs. In such cases, it becomes challenging to pinpoint the exact source code used. Furthermore, even for the same component and version, differences in SBOM types can lead to discrepancies: Source SBOMs do not provide the additional build information, while Build SBOMs include redundant details that are absent from the delivered binary component. Therefore, it is necessary to retain the exact source code information used during the build that is included in the binary component.

5.3.3 Improvement Measures

To improve supply chain transparency, the SBOM Document author can attach source code information for binary components as follows:

- Add the source code information for a binary component as a separate source code component in the SBOM Document and manage its details. For example, SPDX allows linking binary components and source code components using relationships such as GENERATED_FROM, DESCENDANT_OF, or CONTAINS (see also [Section 5.8](#)).
- CycloneDX provides a “pedigree” attribute for holding source code details, allowing the SBOM Document to include direct source code and patch information for the binary component.

The source code information should include, when available:

- The source URL (this may be the URL of a tar.xz file or a version control system URL).
- A hash value of the source code, with a clear indication of what the hash covers and at which level it was computed. Specifically:
 - For archive files (e.g., .tar.xz): provide the hash algorithm (e.g., SHA-512) and confirm that the hash applies to the archive file as a whole, not to individual files within it.
 - For version control systems: provide the commit hash, which unambiguously identifies the exact revision of the source tree.
 - Alternatively, a SWHID (Software Heritage Identifier) may be used, as it encodes both the object type and scope explicitly.

In all cases, the hash entry should specify: (1) the hash algorithm, (2) the hash value, and (3) the scope of the hash (e.g., archive-level, file-level, or commit-level), so that recipients can reliably reproduce and verify the hash.

- If modifications were made during the build process, corresponding patch information should be included alongside the source code details.

5.3.4 Risks and Considerations

Verifying the validity of the source code information attached by the SBOM Document provider to binary components is challenging for SBOM Document recipients.

One potential method of verification is for the recipient to rebuild the binary and confirm that it matches. However, to do so, the SBOM Document provider must supply additional build environment information and ensure that the build is reproducible.

5.4 Component Identification Issues Affecting Vulnerability Handling and License Compliance

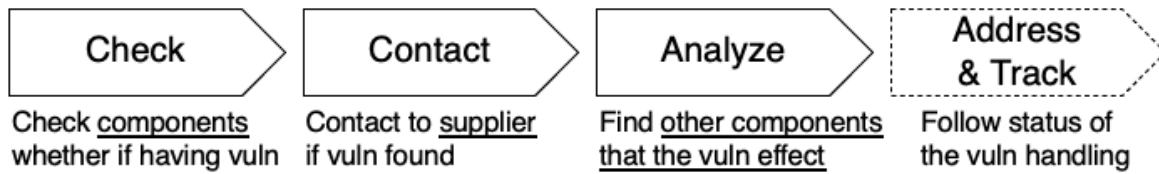
5.4.1 Issue Overview

SBOM is expected to enhance vulnerability handling by providing:

- Component identification information that can be cross-referenced with vulnerability databases to determine if a vulnerability exists
- Supplier information for reporting to remediate found vulnerabilities
- Information of inter-component dependencies to assess how a vulnerability in one component might affect others

However, these elements are not always captured accurately or comprehensively, leading to:

- Inconsistent naming and identifier usage that hampers effective correlation with vulnerability data
- Inadequateness or ambiguity of supplier information reducing the ability to follow up on vulnerability reports
- Incomplete description of dependency relationships risking oversight of cross-component impacts



In addition, detailed information and status of vulnerabilities, if available, can be also described in formats such as VEX (Vulnerability Exploitability eXchange) and associated with SBOM to be used for their remediation and tracking of changes.

5.4.2 Detailed Description

1. When relying on component names and version numbers for identification, inconsistencies such as label variations (as highlighted in [Section 5.1](#)) can occur, leading to missed detection of vulnerabilities. Even the inclusion of unique identifiers like PURLs or CPE names does not guarantee a match in vulnerability databases, since those identifiers may not be present there.
2. SBOM Document generated by automated tools often fall short in providing thorough supplier information. The supplier fields may be omitted or described in a way that does not clearly identify the organization or individual responsible – for instance, sometimes only a name is provided without accompanying contact information such as an email address. This lack of precise supplier data complicates vulnerability reporting and follow-up actions.
3. The completeness of inter-component dependency information depends heavily on the tools, settings, and the technologies (programming languages or development environments) used. This can result in an incomplete mapping of dependency relationships, which in turn may lead to an underestimation of how vulnerabilities in one component can impact others within the software product.

This detailed breakdown underscores that despite SBOM Document's potential in improving vulnerability handling, issues with inconsistent component identification and incomplete data often limit its effectiveness.

5.4.3 Improvement Measures

1. Refer to Section 5.1 and use the naming conventions employed by vulnerability databases such as Open Source Vulnerabilities (OSV) and National Vulnerability Database (NVD) to record the component name, and include the corresponding unique software identifiers (e.g., a PURL for OSV or a CPE name for NVD) within the component information.
2. Ensure that the supplier information identifies a real, publicly recognized entity or individual by verifying its existence and providing clear contact details such as the name, a valid email address, or a URL.
3. Utilize tools that can extract inter-component dependency information from package managers to generate a comprehensive SBOM Document, and when possible, use

build tools capable of producing a Build SBOM according to CISA's SBOM Type Classification.

5.4.4 Risks and Considerations

1. Although unique identifiers such as PURLs or CPE names can assist with component identification, their support is limited because only some databases like OSV support PURLs while others like NVD may not, and different databases might list the same component using slightly varying CPE names. Moreover, dynamically loaded components at runtime may not be captured in the SBOM, which further compromises the accuracy of vulnerability correlation.
2. Since a component's supplier may change over time due to mergers, business closures, or shifts in support responsibilities, it is essential to verify and update supplier information on a regular basis.
3. Automatically extracted dependency information, especially when derived from static analysis or less comprehensive techniques rather than robust build tools, may not accurately capture the actual runtime inter-component relationships, particularly in cases where components are dynamically loaded, leading to gaps in the overall depiction of dependencies within the SBOM Document.

5.5 Establishing a Tamper Detection and Change Management System

5.5.1 Issue Overview

In SBOM operations, the SBOM Document may be updated on a different schedule than the provided software itself, and its content may contain errors. However, there is currently no sufficient mechanism to detect modifications to the SBOM Document, making it difficult to guarantee consistency with the software.

5.5.2 Detailed Description

- Manual modifications leading to data inconsistencies
When the SBOM Document is later edited or supplemented by hand, the resulting document may no longer match the original software configuration, risking unintended changes such as the addition of unneeded component information.
- Lack of version control causing opaque modification history
Without a system to record changes, it becomes impossible to trace who modified the SBOM Document, when, why, or how, making it difficult to distinguish between intentional tampering and routine updates, and preventing rollback to previous, trusted versions.
- Inconsistent formats and missing information
When SBOM Documents are automatically generated by multiple tools, differences in format and content can result in the loss of critical details – such as dependency information – during integration or editing, potentially leading to incomplete security assessments.

5.5.3 Improvement Measures

- **Unique identification and digital signatures**
Every SBOM Document, regardless of format, should carry a unique identifier that is distinct from any digital signature. Upon generation, a digital signature or hash value should be immediately appended to enable later detection of any modifications.
- **Change history management**
SBOM Documents should be treated as immutable. Any modification should result in the creation of a new SBOM Document with a new unique identifier, rather than overwriting the existing one. For example, where supported by the format, relationships such as `AMENDED_BY` or `ANCESTOR_OF` in SPDX should be used to link the updated document to its previous version. All changes should be recorded and tracked using a version control system.
- **Automated verification**
Implement tools that automatically check the integrity of digital signatures or hash values.
- **Automated monitoring**
Establish a tamper-detection process that regularly verifies signatures and issues alerts when abnormalities are detected.
- **Update and redistribution policies**
Formalize procedures for regularly updating the SBOM Documents, reapplying signatures, and automatically redistributing the revised document. Any distribution constraints applicable to the SBOM Document should be explicitly identified and communicated to recipients as part of these procedures.
- **Audit and review framework**
Complement automated measures with periodic reviews, log audits, and manual final verifications.

5.5.4 Risks and Considerations

- **Key management risks**
Be alert to the leakage or expiration of signing keys and certificates; implement proper key management and revocation procedures.
- **Increased operational burden**
Introducing new processes may necessitate changes to development environments and incur additional team training costs.
- **Inter-tool compatibility issues**
Differences in SBOM tool formats can compromise the consistency of hash values and signatures.
- **Minimizing manual intervention**
Maximize automation to eliminate human errors, limiting manual verification to the minimum required.

5.6 Clarifying the Scope of Descriptions and Defining Accountability

5.6.1 Issue Overview

SBOM documents largely rely on the provider's discretion, resulting in inconsistent comprehensiveness and accuracy of the required information. Consequently, essential data may be missing across the software supply chain, increasing the risk of confusion in managing security and license compliance.

5.6.2 Detailed Description

Various guidelines and specifications exist, yet the criteria for including dependent components in an SBOM and the level of detail required for each component remain unclear.

- Only a limited set of necessary details, such as the scope of software dependencies and version information, may be included.
- It is often unclear who is accountable for each component, as contact information and management responsibilities are not clearly defined. This lack of clarity hampers rapid response when issues arise.

Consequently, incomplete information within the SBOM can prevent a full understanding of vulnerabilities and license-related risks across the entire software supply chain, potentially impeding effective security measures.

5.6.3 Improvement Measures

- Establish clear, standardized criteria for the items that need to be included in an SBOM Document across the entire software supply chain, with particular emphasis on clarifying the scope of dependencies.
- Clearly define accountability for each item by including specific responsible individuals and contact details within the document.

When considering the software supply chain, the supplier field should be treated as the primary field for identifying the party accountable for the component, as it most directly represents the entity responsible for its distribution and support across each tier of the supply chain.

When recording contact information, note that available fields differ between formats. In SPDX, the relevant fields are **supplier** and **originator**, while in CycloneDX, **supplier**, **manufacturer**, and **authors** are available. Among these, the **supplier** field should be prioritized in both formats, as it most clearly indicates the party bearing accountability for the component at each point in the supply chain. The **originator** field in SPDX and the **manufacturer** field in CycloneDX may be used to record the entity that originally created the component, which may differ from the supplier in cases involving repackaging or redistribution through multiple supply chain tiers.

Where the existing fields are insufficient to clearly express who holds accountability across the supply chain, for example when distinguishing between legal responsibility and technical contact at different tiers, a Comment field should be used as a supplement to explicitly state the accountable party and the nature of their responsibility.

This approach will enable effective information sharing and rapid response when issues arise. Following diagrams are the software supply chain as an illustrative example.

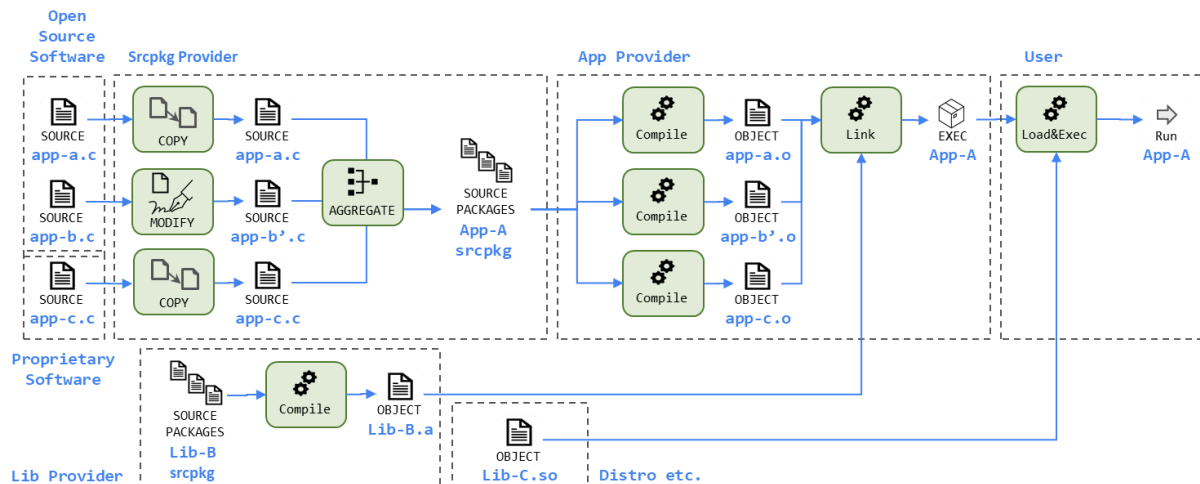


Fig.5-7.1 Entire software supply chain

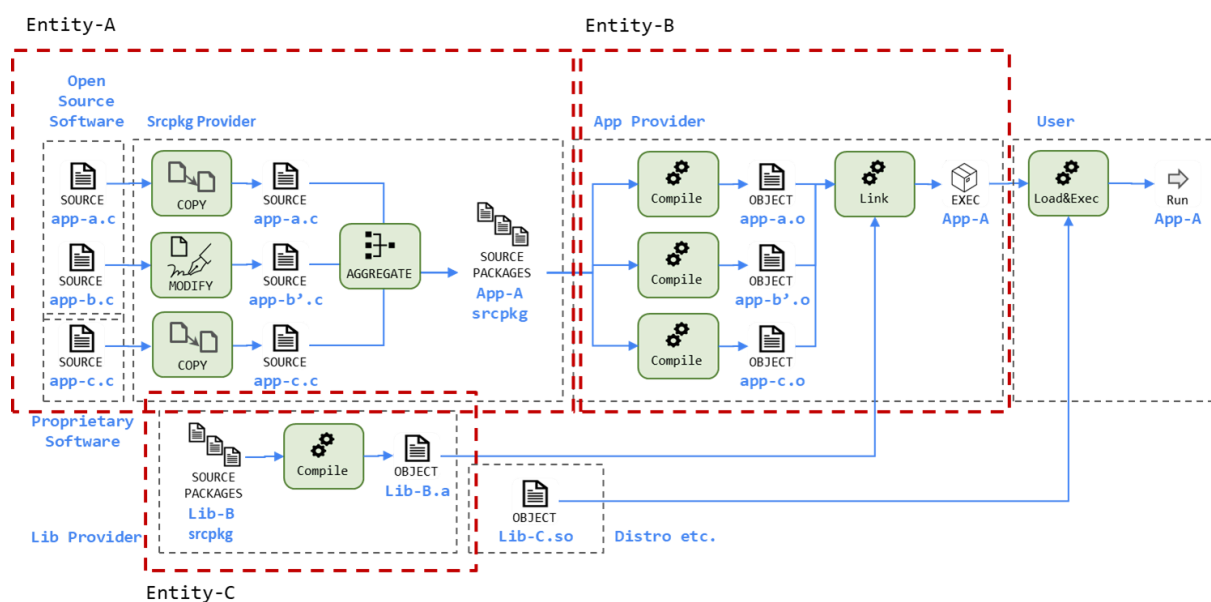


Fig.5-7.2 Three entities for "App-A SBOM Document" provided to "User".

"Entity-B" is generally responsible for direct vulnerability response and OSS license compliance for "User". There is no requirement to provide "User" with the contact details of "Entity-A" and "Entity-C"; however, "Entity-B" must know the proper contact information for "Entity-A" and "Entity-C". Consequently, the SBOM Document provided by "Entity-A" and "Entity-C" must include their respective appropriate contact details.

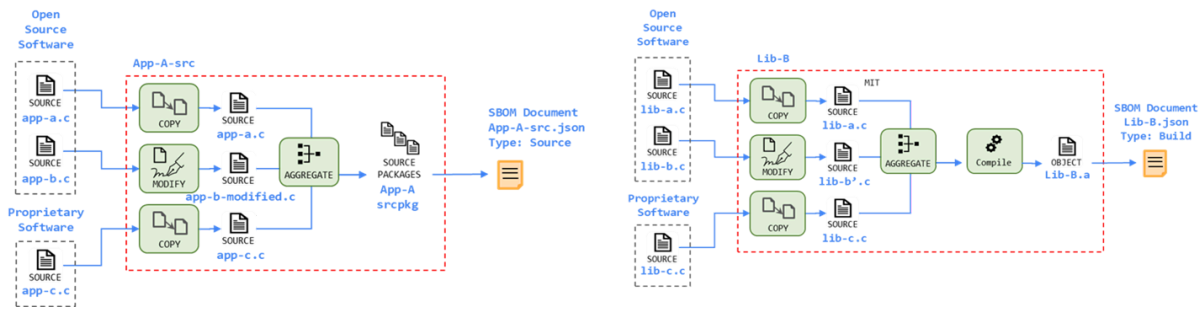


Fig.5-7.3 Independent SBOM Document Creation by Entity-A and Entity-C

Furthermore, if 'Entity-B' is aware that 'App-A' has a dynamic dependency on 'Lib-C.so' but does not know from whom 'Lib-C.so' is actually distributed, it is essential that the SBOM Document for 'App-A' explicitly indicates this dependency. Additionally, the SBOM Document for "App-A" must clearly indicate that its distribution does not include "Lib-C.so" and that detailed information, such as the version number, about "Lib-C.so" remains unavailable, classifying it as "Known Unknown".

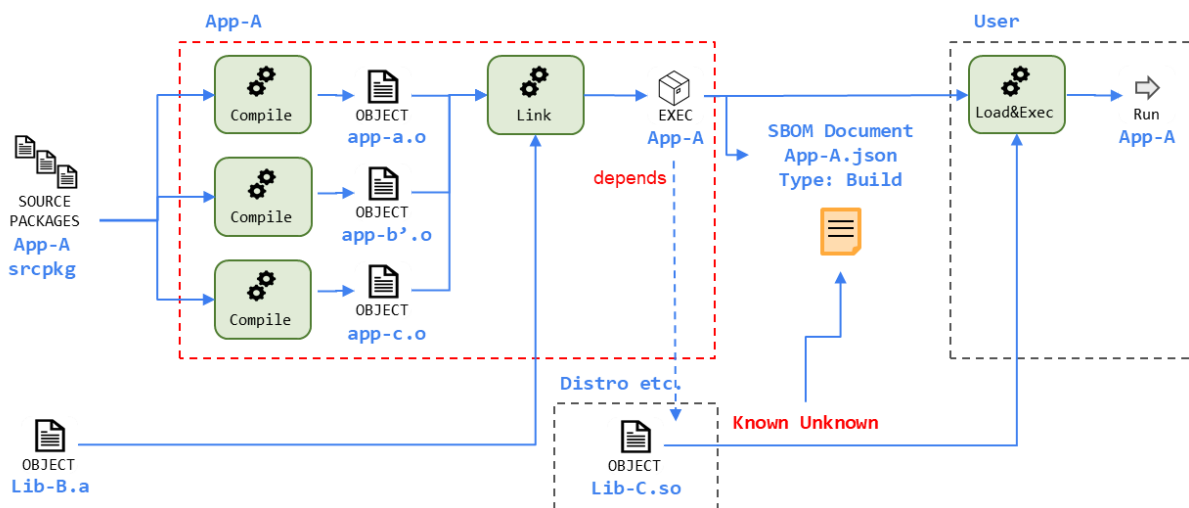


Fig.5-7.4 "Known Unknown" for Dynamic runtime dependencies

5.6.4 Risks and Considerations

- Delays in updating the SBOM Documents in response to software version upgrades or changes in dependencies may result in decisions being made based on outdated information.
- A lack of uniform criteria across the supply chain can lead to inconsistencies among SBOM Documents, causing confusion.
- If the responsible parties or contact information are not clearly specified or are recorded incorrectly, the ability to respond swiftly during issues may be compromised, thereby increasing security risks.

5.7 Unified Expression of Component Relationships

5.7.1 Issue Overview

Inconsistent depiction of relationships between components (e.g., dependency, containment, derivation) may hinder accurate automated analysis and risk assessment.

5.7.2 Detailed Description

SBOM is used to list not only the direct software components but also the libraries, development tools, and developers as SBOM elements, and to describe the relationships among them. However, due to the varying complexity and scale of software, several challenges arise in describing these relationships:

1. The method for describing relationships between components (e.g., depends, contains, generates) varies significantly, and the selected notation differs among individuals, leading to confusion.
2. The notations used are ambiguous, as a single keyword may be applied to different relationships. This ambiguity causes interpretation differences between providers and recipients, potentially impacting risk assessment.

5.7.3 Improvement Measures

Although various keywords may be used to describe relationships, it is recommended that the following primary relationships be consistently included in the SBOM Document and uniformly defined across the entire supply chain.

1. Keywords for describing the current state of a component
 - a. “contains” / “composition-assemblies”
Indicates that a component includes or is composed of another.
 - b. “dependsOn” / “composition-dependencies”
Indicates that a component depends on or requires another.
2. Keywords for describing the origin of a component
 - a. “generatedFrom” / “components-pedigree”
Indicates that a component was generated (replicated, modified, or built) from another.

5.7.4 Risks and Considerations

These recommendations allow the inclusion of additional relationship keywords alongside the primary relationships, provided all terms are coordinated in advance to avoid confusion.

5.8 Interoperability and Flexibility Among Tools

5.8.1 Issue Overview

Different SBOM generation and analysis tools, such as ORT (OSS Review Toolkit), FOSSology, or ScanCode etc., may produce outputs with variations that complicate integration in a unified processing workflow.

Furthermore, optional fields such as “comment” may carry information critical to specific operational contexts, for example, build options or provenance remarks agreed upon between supplier and recipient. However, such information is frequently lost when SBOM Documents are processed, merged, or transformed by tools. Without prior agreement between the supplier and recipient on which fields are to be preserved and how they are to be interpreted, relying on these fields for operational purposes can lead to inconsistencies and workflow failures.

5.8.2 Detailed Description

Many software tools designed for generating and managing SBOM Document are typically developed with interoperability in mind. However, in practical settings, there have been few comprehensive evaluations of the compatibility across these various tools, rendering their true degree of interoperability uncertain.

Moreover, even when compatibility exists, there is generally insufficient documentation on the integration process, which consequently forces users to resort to trial-and-error methods to achieve effective configurations.

Certain SBOM fields, such as Comment, are optional and intended to carry supplementary information. In practice, however, the content of these fields is often lost or discarded when SBOM Documents pass through tools for processing, merging, or format conversion. This problem is compounded by the fact that the meaning and intended use of such fields may vary between the supplier and the recipient. When a supplier records operationally significant information, such as license analysis notes, provenance remarks, or exception justifications, in a Comment field without prior agreement with the recipient, that information may be silently dropped or misinterpreted downstream. As a result, operational decisions that depend on such information can fail or produce inconsistent outcomes, particularly in multi-tier supply chains where SBOM Documents are handed off across organizational boundaries.

5.8.3 Improvement Measures

When employing a mixed environment of tools for handling SBOM Documents, manual adjustments such as value conversions and schema modifications are often unavoidable. To alleviate these issues, it is effective to designate a primary format at the operational level, taking into account the following criteria:

- Existing assets
- The extent of compatibility across a wide range of tools
- The proficiency of the personnel responsible

SBOM Documents received from upstream sources should retain identifiers such as PURL and generator information. Format transition procedures should be standardized through configuration files and minimal sample setups, and syntactic validation (such as with JSON Schema) should be automated in continuous integration environments where possible.

For optional fields such as Comment, suppliers and recipients should establish prior agreement on which fields must be preserved during tool-based processing and how their content is to be interpreted, as such information is frequently lost during merging or format conversion.

To identify and address interoperability gaps, it is strongly encouraged to produce SBOM Documents in both SPDX and CycloneDX formats using multiple tools and the latest version of each standard. A representative process would involve producing at least four SBOM Documents, with two tools each generating both a SPDX and a CycloneDX output. The resulting SBOM Documents should be systematically compared to identify discrepancies in field mapping, value representation, and the preservation of optional fields such as Comment. The findings from such comparisons should be documented and shared with the relevant tool development and standards communities to drive concrete improvements in interoperability. Looking ahead, tool developers, format stewards, and the wider SBOM community should engage in active dialogue to establish common interoperability requirements and resolve compatibility issues across supply chains.

5.8.4 Risks and Considerations

1. **Semantic Loss During Format Conversion**

Even when syntactic compatibility between SPDX and CycloneDX is achieved, information expressed in one format cannot always be accurately reproduced in the other due to structural differences in their respective fields. This risk is particularly pronounced for optional fields such as Comment, where information loss or unintended changes in meaning during format conversion cannot be fully eliminated.

2. **Maintaining Agreements Across Multi-tier Supply Chains**

Even when operational agreements are established with direct trading partners, there is no guarantee that equivalent agreements are maintained further upstream or downstream. The longer and more complex the supply chain, the more difficult it becomes to ensure consistent preservation and interpretation of fields across all parties involved.

3. **Keeping Pace with Evolving Specifications**

Both SPDX and CycloneDX specifications are subject to continuous revision. When tool implementations or organizational procedures fail to keep pace with these changes, interoperability issues are likely to recur. Even as community-level discussions on compatibility progress, whether each tool implements the resulting agreements promptly and accurately remains a separate concern, making ongoing validation and update cycles indispensable.

6. Compliance Elements for SPDX and CycloneDX in Relation to Legislation and Guidelines

This chapter provides a comprehensive analysis of the compliance elements associated with various guidelines within the frameworks of SPDX and CycloneDX. It is organized into sections that detail each guideline's requirements, categorizing them as mandatory or recommended. For each guideline, the chapter outlines the key elements, including their names, associated data fields, and supplementary remarks. Emphasis is placed on establishing a clear mapping between the guideline elements and the corresponding data fields defined in SPDX and CycloneDX. This structured approach facilitates cross-referencing and ensures consistency in documenting compliance requirements across multiple frameworks.

6.1 References

- SPDX v2.2+
<https://spdx.github.io/spdx-spec/v2.3/how-to-use/#k2-satisfying-ntia-minimum-elements-for-an-sbom-using-spdx>
- SPDX v3.0
<https://github.com/spdx/using/blob/main/docs/comply-with-norms.md#satisfying-ntia-minimum-elements-for-an-sbom-using-spdx--us-executive-order-14028>
- CycloneDX v1.7
https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-SBOM-en.pdf
- NTIA - The Minimum Elements For a Software Bill of Materials (SBOM)
<https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>
- CISA - 2025 Minimum Elements for a Software Bill of Materials (SBOM)
<https://www.cisa.gov/resources-tools/resources/2025-minimum-elements-software-bill-materials-sbom>
- BSI TR-03183 Cyber Resilience Requirements for Manufacturers and Products Part 2: Software Bill of Materials (SBOM) Version 2.1.0
“8.2 Mapping of the individual data fields”: Cyber Resilience Requirements for Manufacturers and Products - Part 2: Software Bill of Materials (SBOM) Version 2.1.0
https://www.bsi.bund.de/EN/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Technische-Richtlinien/TR-nach-Thema-sortiert/tr03183/TR-03183_node.html
- OpenChain Telco SBOM Guide Version 1.1
https://github.com/OpenChain-Project/Telco-WG/blob/main/OpenChain-Telco-SBOM-Guide_EN.md

6.2 Cross-Regulation Comparison Table

This table consolidates all element requirements from the four standards covered in this chapter. It enables implementers to identify which elements satisfy multiple regulations simultaneously and to design a single SBOM that meets all applicable obligations.

Category	Element	NTIA 2021	CISA 2025	BSI TR-03183	OpenChain Telco Guide	Notes / Points of Attention
SBOM Document Metadata	SBOM Author / Creator of SBOM	SHALL	SHALL	SHALL	SHALL	Different names across standards: 'Author of SBOM Data' (NTIA), 'SBOM Author' (CISA), 'Creator of the SBOM' (BSI). Contact details (email or URL) should be included where possible.
	Timestamp	SHALL	SHALL	SHALL	SHALL	
	Tool Name	—	SHALL	—	SHALL	New mandatory field added in CISA 2025; absent from NTIA 2021. Enables recipients to assess SBOM data quality and reproducibility. Tool version SHOULD also be recorded.
	Generation Context / Lifecycle Phase	SHOULD	SHALL	—	SHOULD	Upgraded from SHOULD (NTIA 2021) to SHALL (CISA 2025). Records whether the SBOM was generated before build, during build, or after build. BSI does not address this field explicitly. Build-time SBOMs are considered the most accurate.
	Unique ID for SBOM Document	SHALL	SHALL	SHALL	SHALL	BSI names this 'SBOM-URI'. In CycloneDX a BOM-Link (urn:cdx:{serialNumber}/{version}) is used for cross-referencing. Must be globally unique.
	License for the SBOM Document	—	—	—	SHALL	Telco guide-specific: SPDX v2 requires the DataLicense field (CC0-1.0). Not addressed by NTIA, CISA, or BSI.
Software Package Information	Software Producer / Supplier Name	SHALL	SHALL	SHALL	SHALL	CISA 2025 renamed 'Supplier Name' (NTIA) to 'Software Producer' to avoid confusion with distributors. BSI additionally requires a URL or email contact address (mandatory, not optional).

	Component Creator / Originator	—	—	SHALL	—	BSI-specific field. Distinguishes the entity that wrote the code from the entity that supplies/distributes it. Must include a URL or email address. Not present in NTIA or CISA.
	Component Name	SHALL	SHALL	SHALL	SHALL	Naming conventions differ across vulnerability databases (OSV, NVD). Suppliers and recipients should agree on which naming convention to follow (see Section 5.1).
	Component Version	SHALL	SHALL	SHALL	SHALL	
	Software Identifiers (PURL / CPE / SWHID)	SHALL	SHALL	SHALL	SHOULD	Different names: 'Other Unique Identifiers' (NTIA), 'Software Identifiers' (CISA). PURL is supported by OSV; CPE names are used by NVD. Recording both improves cross-database coverage. Telco recommends PURL specifically. BSI requires CPE, PURL, or SWID.
	Component Hash	SHOULD	SHALL	SHALL	SHOULD	Upgraded from SHOULD (NTIA) to SHALL (CISA 2025). BSI explicitly requires the hash of the deployable artifact (binary); source code hash is a separate optional field. SHA-256 or stronger is recommended.
	Filename of the Component	—	—	SHALL	—	BSI-specific. Mandatory for traceability of deployable artifacts. Not addressed by NTIA, CISA, or Telco.
	Package Download / Distribution URL	—	—	SHALL	SHALL	Required by both BSI and Telco, but not by NTIA or CISA. BSI also separately requires the URI of the deployable form of the component.
	Source Code URI	—	—	SHALL	—	BSI-specific. Enables traceability from binary back to source. Particularly important for CRA compliance and Open Source license verification. See also Section 5.3 on source code transparency.
	File Type Properties (Executable /	—	—	SHALL	—	BSI-specific. Classifies the type of each deployable artifact (executable binary, archive, or structured data such

	Archive / Structured)					as container/firmware). Required for CRA-aligned artifact traceability.
License Information	Declared License	SHOULD	SHALL	SHALL	SHALL	Upgraded from SHOULD (NTIA 2021) to SHALL (CISA 2025). Use SPDX License Expressions for consistency. For proprietary licenses, include contact information alongside the Proprietary Software indicator.
	Concluded License	SHOULD	SHALL	SHALL	SHALL	Upgraded from SHOULD (NTIA 2021) to SHALL (CISA 2025). If the license cannot be determined, mark with 'NOASSERTION' rather than omitting the field. BSI calls this 'Distribution licenses'.
	Effective Licence	—	—	MAY	—	BSI-specific optional field. Records the license that effectively applies after considering dual-licensing, dependency relationships, and distribution context. Not addressed by other standards.
	Copyright Text	—	—	—	SHALL	Required by Telco guide (and mandatory in SPDX v2 format). Not required by NTIA, CISA, or BSI at the SBOM element level.
Relationship Information	Dependency Relationship	SHALL	SHALL	SHALL	SHALL	All standards require this. NTIA/CISA: document must DESCRIBE at least one package and record CONTAINS relationships. BSI specifically requires 'contains' or 'dependsOn' relationship types. See Section 5.7 for guidance on unified relationship expressions.
	Other Component Relationships	SHOULD	—	SHOULD	—	NTIA recommends recording additional relationships such as GENERATES, ANCESTOR_OF, and VARIANT_OF. CISA and Telco do not address these. See Section 5.7 for recommended keywords.

6.3 NTIA - The Minimum Elements For a Software Bill of Materials (SBOM)

6.3.1 Required Elements

Req.	Element Name	SPDX v2.2+	SPDX v3.0	CycloneDX v1.7
SHALL	Supplier Name	(7.5) PackageSupplier	Core/Classes/Artifact.suppliedBy	bom.metadata.supplier bom.components[].supplier
SHALL	Component Name	(7.1) PackageName	Software/Classes/Package.name	bom.components[].name
SHALL	Version of the Component	(7.3) PackageVersion	Software/Classes/Package.packageVersion	bom.components[].version
SHALL	Other Unique Identifiers	(7.2) Package SPDX Identifier (6.5) SPDX Document Namespace (7.21) ExternalRef: PURL/CPE	Software/Classes/SoftwareArtifact.contentIdentifier Software/Classes/Package.packageUrl Core/Classes/Element.externalIdentifier	bom.components[].cpe, .purl, .swid
SHALL	Dependency Relationship	(11.1) Relationship: CONTAINS, DESCRIBES Document must DESCRIBE ≥ 1 package	Core/Classes/Relationship	bom.dependencies[]
SHALL	Author of SBOM Data	(6.8) Creator	Core/Classes/CreationInfo.createdBy	bom.metadata.author
SHALL	Timestamp	(6.9) Created	Core/Classes/CreationInfo.created	bom.metadata.timestamp
SHOULD	Hash of the Component	(7.10) PackageChecksum	Core/Classes/Element.verifiedUsing	bom.components[].hashes[]

SHOULD	Lifecycle Phase	N/A → (6.10) CreatorComment (workaround)	Software/Sbom/sbomType	bom.metadata.lifecycles[]
SHOULD	Other Component Relationships	(11.1) GENERATES, ANCESTOR_OF, VARIANT_OF...	Core/Classes/Relationship (various types)	bom.components[].pedigree
SHOULD	License Information	(7.13) PackageLicense Concluded (7.15) PackageLicense Declared	RelationshipType::hasConcludedLicense RelationshipType::hasDeclaredLicense	bom.components[].licenses[]

6.4 CISA - 2025 Minimum Elements for a Software Bill of Materials (SBOM)

6.4.1 Required Elements

Req.	Element Name	SPDX v2.2+	SPDX v3.0	CycloneDX v1.7
SHALL	SBOM Author	(6.8)Creator	Core/Classes/CreationInfo.createdBy	bom.metadata.author
SHALL	Software Producer	(7.5)PackageSupplier	Core/Classes/Artifact.suppliedBy	bom.components[].supplier
SHALL	Component Name	(7.1)PackageName	Software/Classes/Package.name	bom.components[].name
SHALL	Component Version	(7.3)PackageVersion	Software/Classes/Package.packageVersion	bom.components[].version
SHALL	Software Identifiers	(7.2)Package SPDX Identifier (6.5)SPDX Document Namespace (7.21)ExternalRef: PURL/CPE	Software/Classes/SoftwareArtifact.contentIdentifier	bom.components[].cpe, .purl, .swid
SHALL	Component Hash	(7.10)PackageChecksum	Core/Classes/Element.verifiedUsing	bom.components[].hashes[]
SHALL	License	(7.13)PackageLicenseConcluded (7.15)PackageLicenseDeclared	RelationshipType::hasConcludedLicense RelationshipType::hasDeclaredLicense	bom.components[].licenses[]
SHALL	Dependency Relationship	(11.1) Relationship: CONTAINS, DESCRIBES Document must DESCRIBE ≥ 1 package	Core/Classes/Relationship	bom.dependencies[]
SHALL	Tool Name	(6.8) Creator: Tool keyword	Core/Tool	bom.metadata.tools

SHALL	Timestamp	(6.9) Created	Core/Classes/CreationInfo.c reated	bom.metadata.timestamp
SHALL	Generation Context	(6.9) Created → model-definition	Software/Sbom/sbomType (via model-definition)	bom.metadata.lifecycles[]

6.4.2 Table of Minimum Elements Data Fields

Data Field	Description
SBOM Author	The name of the entity that creates the SBOM data for this component.
Software Producer	The name of an entity that creates, defines, and identifies components.
Component Name	The name assigned by the Software Producer to a software component.
Component Version	Identifier used by the Software Producer to specify a change in software from a previously identified version.
Software Identifiers	Identifier(s) used to identify a component or serve as a look-up key for relevant databases.
Component Hash	The cryptographic value generated from taking the hash of the software component.
License	The license(s) under which the software component is made available.
Dependency Relationship	The relationship between two software components, specifically noting that Software X includes Component Y or that Component A is largely derived from Component B.
Tool Name	The name of the tool used by the SBOM Author to generate the SBOM.
Timestamp	Record of the date and time of the most recent update to the SBOM data.
Generation Context	The relative software lifecycle phase and data available at the time the Software Producer generated the SBOM (before build, during build, after build).

6.5 BSI TR-03183 Cyber Resilience Requirements for Manufacturers and Products Part 2: Software Bill of Materials (SBOM) Version 2.1.0

6.5.1 Required Elements

TR-03182 version 2.1.0 does not allow SPDX v2.

Req.	Element Name	SPDX v3.0	CycloneDX v1.7
SHALL	Creator of the SBOM	Core/Classes/CreationInfo.createdBy	bom.metadata.manufacturer[].url XOR bom.metadata.manufacturer[].contact[].email
SHALL	Timestamp	Core/Classes/CreationInfo.created	bom.metadata.timestamp
SHALL	Component creator	Software/Classes/Package.originatedBy	bom.components[].manufacturer[].url XOR bom.components[].manufacturer[].contact[].email
SHALL	Component name	Software/Classes/Package.name	bom.components[].name (+ optionally .group)
SHALL	Component version	Software/Classes/Package.packageVersion	bom.components[].version
SHALL	Filename of the component	Software/Classes/File.name	bom.components[].properties[name=bsi:component:filename]
SHALL	Dependencies on other components	Core/Classes/Relationship (contains / dependsOn)	bom.components[].components bom.dependencies[] bom.compositions.assemblies[] / .dependencies[]
SHALL	Distribution licenses	Core/Classes/Relationship (hasConcludedLicense)	bom.components[].licenses[].expression [acknowledgement=concluded]
SHALL	Hash value of deployable component	Software/Classes/File.verifiedUsing	components[].externalReferences[].hashes[] [type=distribution]

SHALL	Executable property	Software/Classes/File.additionalPurpose=[executable]	bom.components[].properties[name=bsi:component:executable]
SHALL	Archive property	Software/Classes/File.additionalPurpose=[archive]	bom.components[].properties[name=bsi:component:archive]
SHALL	Structured property	Software/Classes/File.additionalPurpose=[container firmware]	bom.components[].properties[name=bsi:component:structured]
SHALL	SBOM-URI	Software/Classes/Sbom.spdxId	bom.serialNumber (BOM-Link: urn:cdx:{serialNumber}/{version})
SHALL	Source code URI	Software/Classes/SoftwareArtifact.externalRef (externalRefType=SourceArtifact)	bom.components[].externalReferences[type=source-distribution].url
SHALL	URI of deployable form of component	Software/Classes/File.externalRef (externalRefType=binaryArtifact) [under discussion]	bom.components[].externalReferences[type=distribution].url
SHALL	Other unique identifiers	Software/Classes/Package.externalIdentifiers (cpe22, cpe23, swid, packageURL)	bom.components[].cpe, .swid, .purl
SHALL	Original licences (declared)	Core/Classes/Relationship (hasDeclaredLicense)	bom.components[].licenses[].expression [acknowledgement=declared]
<i>MAY</i>	Effective licence	Core/Classes/Relationship (type=other, comment=hasEffectiveLicense)	bom.components[].properties[name=bsi:component:effectiveLicense]
<i>MAY</i>	Hash value of source code	Software/Classes/SoftwareArtifact.verifiedUsing + Relationship (generates)	bom.components.externalReferences[].hashes[] [type=source-distribution]
<i>MAY</i>	URL of security.txt	Software/Classes/Package.externalRef (externalRefType=securityOther)	bom.components[].externalReferences[type=rfc-9116].url

6.6 OpenChain Telco SBOM Guide Version 1.1

6.6.1 Required Elements

The Telco SBOM Guide is primarily based on SPDX v2.2. SPDX v3.0 and CycloneDX equivalents are shown for reference only; they are not formally mandated by the Telco guide. Elements with no direct CycloneDX counterpart are marked with a dash (—).

6.6.1.1 Document Creation Information

Req.	Element Name	SPDX v2.2+	SPDX v3.0	CycloneDX v1.7
SHALL	SPDXVersion	(6.1)SPDXVersion	Core/CreationInfo.specVersion	specVersion
SHALL	DataLicense	(6.2)DataLicense	Core/SpdxDocument.dataLicense	metadata.licenses
SHALL	SPDXID (document)	(6.3)SPDXID	Core/SpdxDocument.spdxId	serialNumber
SHALL	DocumentName	(6.4)DocumentName	Core/SpdxDocument.name	—
SHALL	DocumentNamespace	(6.5)SPDX Document Namespace	Core/SpdxDocument.namespaceMap	—
SHALL	Creator: Organization + Tool	(6.8)Creator: Organization, Tool	Core/CreationInfo.createdBy Core/CreationInfo.createdUsing	metadata.{component.manufacturer authors} metadata.tools
SHALL	Created	(6.9)Created	Core/CreationInfo.created	metadata.timestamp
SHALL	CreatorComment (SBOM Build Information)	(6.10)CreatorComment	Core/CreationInfo.comment	metadata.lifecycles

6.6.1.2 Package Information

Req.	Element Name	SPDX v2.2+	SPDX v3.0	CycloneDX v1.7
------	--------------	------------	-----------	----------------

SHALL	PackageName	(7.1) PackageName	Software/Package.name	metadata.component.name components.name
SHALL	SPDXID (package)	(7.2) Package SPDX Identifier	Software/Package.spdxId	metadata.component.bom-ref components.bom-ref
SHALL	PackageVersion	(7.3) PackageVersion	Software/Package.packageV ersion	metadata.component.version components.version
SHALL	PackageSupplier	(7.5) PackageSupplier	Software/Package.suppliedB y	metadata.component.supplier components.supplier
SHALL	PackageDownloadLocation	(7.7) PackageDownloadLocation	Software/Package.download Location	metadata.component.externalReferences components.externalReferences
SHALL	PackageLicenseConcluded	(7.13) PackageLicenseConcluded	Relationship (hasConcludedLicense)	metadata.component.license s[] {expression, acknowledgement=conclude d}
SHALL	PackageLicenseDeclared	(7.15) PackageLicenseDeclared	Relationship (hasDeclaredLicense)	metadata.component.license s[] {expression, acknowledgement=declared}
SHALL	PackageCopyrightText	(7.17) PackageCopyrightText	Software/Package.copyright Text	metadata.component.copyright components.copyright
SHALL	Relationship: DESCRIBES + CONTAINS	(11.1) Relationship: CONTAINS, DESCRIBES	Relationship (contains, describes)	components.components dependencies
SHOULD	PackageChecksum	(7.10) PackageChecksum	Software/Package.verifiedUs ing	metadata.component.hashe s components.hashes
SHOULD	ExternalRef: PURL (PackageURL)	(7.21) ExternalRef: PACKAGE-MANAGER purl	Software/Package/externalId entifier (packageUrl)	purl

Appendix.1 Acknowledgments

This list acknowledges those who have provided edits, feedback, and comments. The names are arranged in alphabetical order.

We extend our sincere gratitude to all contributors for their generous time, thoughtful insights, and continued dedication to improving this document.

Akihiko Takahashi
Alberto Pianon
Anthony Harrison
Ayumi Watanabe
Fuminobu Takeyama
Gary O'Neill
Itaru Hosomi
Jari Koivisto
Kate Stewart
Kiyoshi Owada
Kouki Hama
Marc-Etienne Vargenau
Nobuyuki Tanaka
Norio Kobota
Philippe Ombredanne
Satoru Koizumi
Shane Coughlan
Tadayuki Osaki
Takashi NINJOUJI
Tsukasa Yobo
Vivek Kumar
Yoshiyuki Ito
Yosuke Hirano
Yumi Tomita
조명식

OpenChain Board members
OpenChain Telco Working Group

Appendix.2 Version History

2026/04/03 version: 2026.04.03 Release